

Programação Orientada a Objetos

Aula 12

João Choma Neto
joao.choma@gmail.com

Chamada está em

- <https://github.com/JoaoChoma/POO>

Classe Abstrata

- Vamos supor que estamos desenvolvendo um jogo e precisamos criar uma classe para representar as entidades do jogo
- Entidades:
 - jogadores, inimigos, objetos coletáveis

Classe Abstrata

- Uma abordagem possível seria criar uma classe abstrata chamada "Entidade" que contém os atributos e métodos que são comuns a todas as entidades do jogo. Por exemplo:

- public abstract class Entidade {
- protected int posX;
- protected int posY;
- protected int velocidade;
- public Entidade(int posX, int posY, int velocidade) {
- this.posX = posX;
- this.posY = posY;
- this.velocidade = velocidade;
- }
- public abstract void mover();
- public void renderizar() {
- // método para renderizar a entidade na tela
- }
- }

Classe Abstrata

- Nesse exemplo, a classe abstrata "Entidade" possui três atributos (posX, posY e velocidade) e dois métodos (mover e renderizar). A classe também possui um construtor que inicializa os atributos.
- Note que o método "mover" é declarado como abstrato, o que significa que ele não é implementado na classe "Entidade", mas sim nas classes filhas que herdam desta classe abstrata

Classe Abstrata

- Isso permite que cada tipo de entidade possa se mover de uma maneira específica
- Ao criar uma classe que herda de "Entidade", é necessário implementar o método "mover", pois ele é abstrato. Por exemplo, podemos criar uma classe "Jogador" que herda de "Entidade":

- public class Jogador extends Entidade {
- public Jogador(int posX, int posY, int velocidade) {
- super(posX, posY, velocidade);
- }
- @Override
- public void mover() {
- // implementação do método mover para o jogador
- }
- }

Classe Abstrata

- Nesse exemplo, a classe "Jogador" herda os atributos e métodos da classe abstrata "Entidade" e implementa o método "mover" de acordo com as especificidades do jogador
- Além disso, como a classe "Entidade" é abstrata, não é possível instanciá-la diretamente, mas sim criar instâncias de suas classes filhas, como "Jogador"

Classe Abstrata

- Dessa forma, a classe abstrata "Entidade" permite que as entidades do jogo compartilhem comportamentos e atributos comuns, enquanto as classes filhas implementam o comportamento específico de cada tipo de entidade
- O uso de classes abstratas também ajuda a evitar a duplicação de código e a aumentar a coesão e o encapsulamento do código.

Atividade classe abstrata

- Entrega: <https://forms.gle/jFg1wYscZNhx3gGU8>

Lista está no drive

Links

- <https://netbeans.apache.org/download/nb17/index.html>
- <https://www.oracle.com/br/java/technologies/downloads/#jdk19-windows>
- <https://github.com/JoaoChoma/POO>

Referências utilizadas

- ASCENCIO, A. F. G. Fundamentos da programação de computadores. 2ª. Edição. São Paulo: Pearson Prentice Hall.
- <https://www.oracle.com/>

Programação Orientada a Objetos Aula 12

João Choma Neto
joao.choma@gmail.com